Sheet2App Design Doc

Nov 7, 2026 · @Jas

Overview

Sheet2App turns an existing financial spreadsheet into a working web app: upload an .xlsx, and get dashboards, expense forms, approvals and alerts in minutes. An LLM infers the workflow the sheet represents and writes a declarative app config; deterministic code, not the model, runs every calculation and approval rule.

Problem. Small businesses, student organizations and small teams run their finances in Excel. Expenses are typed in by hand, formulas break as sheets grow, and approvals happen over email or chat with no audit trail. Dedicated finance platforms fix this, but switching means rebuilding workflows from scratch.

Solution. Sheet2App keeps the spreadsheet as the starting point. It reads the sheet's structure, formulas and cross-sheet references, proposes an app (entities, views, roles, rules), lets the user confirm or adjust it in plain language, then hosts the result.

Example. A student org uploads its event budget. It gets an app where members submit reimbursement requests with receipts, the treasurer approves them, and budget lines update automatically. The treasurer types "Require two approvals for purchases over $200" and the rule takes effect after a preview.

Target users: student organizations, small businesses (1–50 people), and teams inside larger companies that track a budget outside the ERP.

Goals and non-goals

Goals

1. **Upload to working app in under 10 minutes** for a typical budget or expense sheet (⩽ 10 sheets, ⩽ 5,000 rows).
2. **Faithful numbers.** Every total in the generated app matches the original spreadsheet's computed values on import, verified automatically.
3. **No-code customization.** Users change fields, roles, approval rules and alerts through natural language or a simple editor, with a preview before anything goes live.
4. **Trustworthy workflows.** Approvals, budget checks and alerts are executed by deterministic, tested code, with a full audit log.

5. **Less manual entry.** Receipts and invoices are parsed into draft expense records the submitter only confirms.

Non-goals (v1)

- Full accounting: no general ledger, double-entry bookkeeping, tax filing or payroll.
- Moving money: no payments or bank transfers; we record reimbursements, we don't send them.
- Arbitrary Excel fidelity: VBA macros, pivot tables, external data connections and charts are not reproduced (charts are regenerated as dashboard widgets).
- Live two-way sync with the original file; import is one-time, with CSV/XLSX export back out.
- Letting the LLM make financial decisions or compute amounts shown to users.

Personas and core journeys

Three roles cover v1; every generated app maps its users onto them (custom role names are labels on top).

Persona	Example	Needs	Role in app
Owner / admin	Club treasurer, small-business owner	Build and configure the app, set budgets and rules, see everything	Admin
Approver	Club president, department lead	Review requests fast, see budget impact before approving	Approver
Submitter	Club member, employee	Submit an expense with a receipt in under a minute, track its status	Submitter

Journey 1 — Create an app (admin)

1. Upload `Event_Budget_2026.xlsx`.
2. Review the *Detected model* screen: entities found (Budget Lines, Expenses, Members), calculated fields, and inferred workflow, each with a confidence badge.
3. Fix anything wrong by clicking or by typing ("Category is a dropdown, not free text").
4. Preview the app with imported data; the reconciliation check shows every original total matches.
5. Invite members by email and publish.

Journey 2 — Submit and approve (submitter → approver)

1. Member snaps a receipt; fields (vendor, date, amount, category) are pre-filled for confirmation.
2. Rule engine evaluates the request: $240 exceeds $200, so two approvals are required.
3. Approvers are notified, see remaining budget for that line, approve or reject with a comment.
4. On final approval, budget actuals update and the submitter is notified.

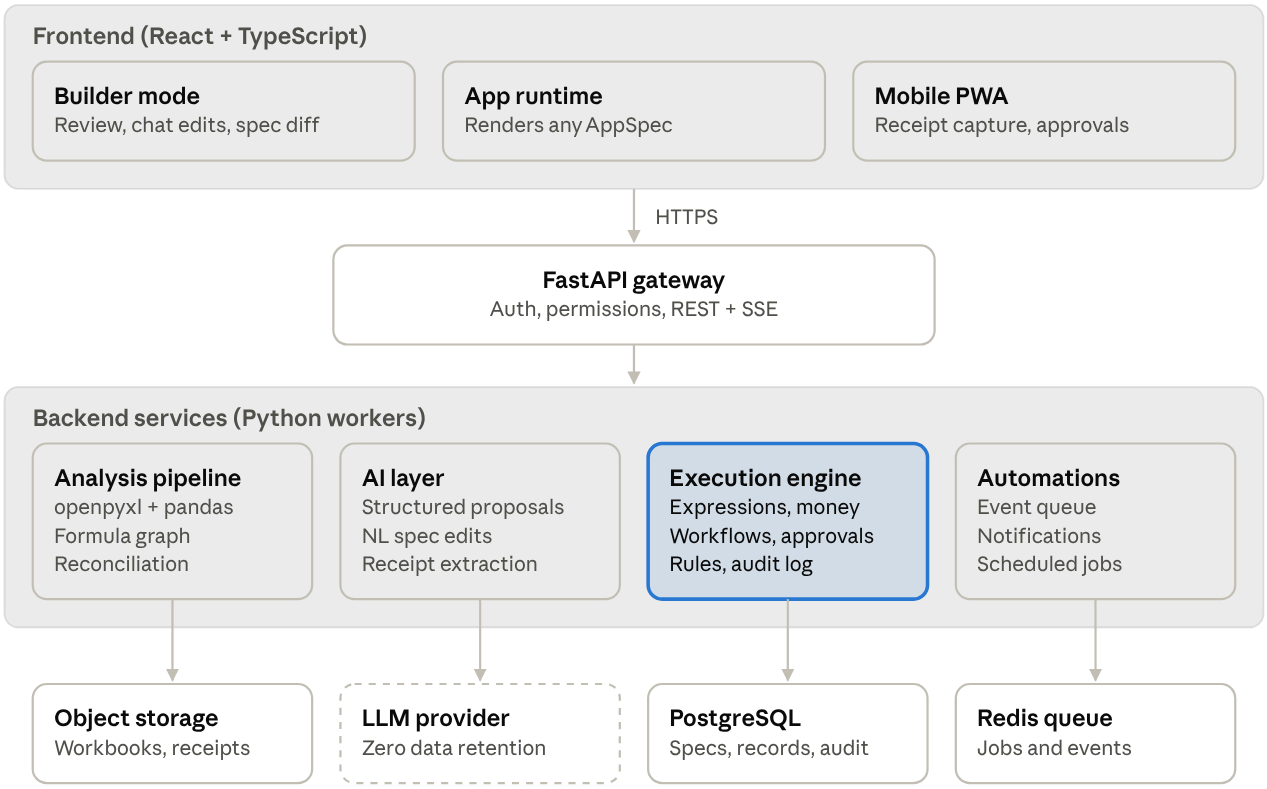
Journey 3 — Change a rule (admin)

1. Admin types "Alert me when any budget line is 90% spent."
2. System shows a diff of the config change and which existing records would be affected.
3. Admin confirms; the change is versioned and can be rolled back.

System architecture

Sheet2App is a single multi-tenant web service: one frontend renders every app, one FastAPI backend serves them, and background workers run the slow AI and automation work.

AI proposes the app; deterministic services run it



The AI layer only writes and edits the spec; every amount and approval comes from the execution engine.

system architecture · frontend, API, 4 backend services, 4 stores

The frontend never talks to the LLM directly. Uploads flow through the analysis pipeline and AI layer to produce a draft spec; once a person confirms it, all runtime reads, writes and approvals go through the execution engine and land in PostgreSQL.

Deployment (v1): containerized services on a managed platform (e.g. AWS ECS/Fargate or Render), managed PostgreSQL, S3-compatible storage, Redis, and a CDN for the frontend. Observability via OpenTelemetry traces, structured logs and per-call LLM cost tracking.

Spreadsheet analysis pipeline

Deterministic parsing does the heavy lifting; the LLM only interprets a compact, pre-digested summary of the workbook and returns schema-validated JSON. Each stage writes its output to the database so a run can be inspected, retried or resumed.

1. **Ingest and sanitize.** Accept .xlsx/.xlsm/.csv up to 25 MB. Strip macros (never executed), external links and embedded objects. Store the original in object storage with a SHA-256 hash.
2. **Structural extraction (openpyxl).** Load twice: `data_only=False` for formulas, `data_only=True` for cached values. Capture per sheet: used range, merged cells, number formats (currency, date, percent), data validation lists, named ranges, Excel Tables, conditional formatting and comments.
3. **Region detection.** Split each sheet into blocks (header row + contiguous data, input forms, summary areas) using heuristics: bold/filled header rows, blank-row separators, Excel Table bounds, formula density. Output: candidate tables with column ranges.
4. **Column profiling (pandas).** For each candidate table: inferred type, null rate, uniqueness, value set (for enums), min/max, and sample values (PII-masked, ≤ 5 per column).
5. **Formula graph.** Parse formulas with a tokenizer (e.g. the `formulas` or `pycel` library) into an AST; build a dependency graph across cells, sheets and named ranges. Collapse copied-down formulas into one column-level expression (`Remaining = Budget - SUMIFS(Expenses[Amount], Expenses[Line], [@Line])`). Flag lookups (`VLOOKUP`, `XLOOKUP`, `SUMIFS`) as candidate relationships between tables.
6. **Workbook digest.** Combine 3–5 into a compact JSON summary, typically 2–10k tokens regardless of row count. Raw rows are never sent wholesale.
7. **LLM inference (structured output + tools).** The model receives the digest and returns a `ModelProposal`: entities, fields, relationships, calculated fields, roles and the inferred workflow (e.g. *reimbursement with approval*), each with a confidence score and a cited source range. It can call read-only tools for more context: `get_range(sheet, a1)`, `get_formula(cell)`, `get_column_profile(table, col)`, `list_dependents(cell)`.
8. **Validation and repair.** The proposal is checked against the JSON Schema and semantic rules (every calculated field resolves to a supported expression; every relationship's keys exist; every role has at least one permission). Failures go back to the model with the error list, up to 2 repair rounds; anything still failing is shown to the user as "needs review".
9. **Formula translation.** Supported Excel functions are compiled into the engine's expression language (see Deterministic execution). Unsupported formulas are kept as imported static values and flagged.

10. **Reconciliation.** Import the data, recompute every calculated field with the engine, and compare to Excel's cached values (tolerance $0.005). The app cannot be published until mismatches are resolved or explicitly accepted.

Model choice. Use a frontier model with native structured outputs for steps 7–8, and a smaller, cheaper model for column-type tie-breaking and labeling. Prompts, schema version and model ID are recorded on every run for reproducibility.

App configuration schema

The app config (`AppSpec`) is the single contract between the AI layer, the backend engine and the frontend renderer. It is declarative, versioned and contains no executable code, so it can be validated, diffed and rolled back.

Section	Defines	Example
`entities`	Tables, fields, types, validation	`Expense { amount: money, date: date, category: enum, receipt: file }`
`relationships`	Foreign keys between entities	`Expense.budget_line → BudgetLine.id`
`computed`	Calculated fields in the expression language	`BudgetLine.remaining = budget - sum(expenses.amount where status = "approved")`
`roles`	Roles and row/field-level permissions	Submitters see only their own expenses
`workflows`	State machines with transitions and guards	`draft → submitted → approved → reimbursed`
`rules`	Approval policies and validations	Two approvals when `amount > 200`
`automations`	Trigger → condition → action	On approve: notify submitter; on 90% spent: email admin
`views`	Pages, forms, tables, dashboard widgets	KPI tiles, budget-vs-actual chart, approval queue

Abridged example:

```
{
  "spec_version": "1.0",
  "entities": {
    "Expense": {
      "fields": {
        "amount":   {"type": "money", "currency": "USD", "required": true,
"min": 0.01},
        "category": {"type": "enum", "options": ["Food", "Venue",
"Supplies"]},
        "budget_line": {"type": "ref", "entity": "BudgetLine"},
        "receipt":  {"type": "file", "accept": ["image/*",
"application/pdf"]}
      },
      "workflow": "reimbursement"
    }
  },
  "workflows": {
    "reimbursement": {
      "states": ["draft", "submitted", "approved", "rejected", "reimbursed"],
      "transitions": [
        {"from": "submitted", "to": "approved", "action": "approve",
         "guard": {"rule": "approval_policy"}}
      ]
    }
  },
  "rules": {
    "approval_policy": {
      "type": "approval",
      "tiers": [
        {"when": "amount <= 200", "approvals": 1, "from_roles":
["Approver"]},
        {"when": "amount > 200",  "approvals": 2, "from_roles": ["Approver",
"Admin"],
         "distinct_approvers": true, "exclude_submitter": true}
      ]
    }
  },
  "automations": [
    {"on": "BudgetLine.updated", "if": "spent / budget >= 0.9",
     "do": {"notify": {"roles": ["Admin"], "template": "budget_threshold"}}}
  ]
}
```

Design rules

- Pydantic models are the source of truth; JSON Schema for LLM structured output and TypeScript types for the frontend are generated from them, so the three never drift.
- Every LLM-produced element carries `provenance` (source sheet/range, confidence) for the review screen.
- Specs are immutable once published; edits create a new version with a migration plan for existing data (add/rename field, change enum options).

Deterministic execution engine

The LLM writes configuration; it never computes a number or decides an approval at runtime. A small, sandboxed expression language and a rules engine in the FastAPI backend execute everything users see.

Expression language

- A typed, side-effect-free subset covering arithmetic, comparisons, `if`, aggregates (`sum`, `count`, `avg`, `min`, `max`) over related records with `where` filters, date functions, and `round`.
- Parsed with a grammar (e.g. Lark) into an AST and type-checked at spec-validation time; no `eval`, no Python execution.
- Excel mapping covers the common finance set first: `SUM`, `SUMIF(S)`, `COUNTIF(S)`, `AVERAGE`, `IF`, `IFERROR`, `ROUND`, `VLOOKUP` / `XLOOKUP` / `INDEX-MATCH` (as relationships), `TODAY`, `EOMONTH`. Anything else is flagged for review.

Money and precision

- Amounts are stored as `NUMERIC(14,2)` in Postgres and `Decimal` in Python; never floats. Rounding is banker's or half-up per spec, applied at defined points.
- One currency per app in v1; multi-currency is a later milestone.

Computed fields

- Recomputed incrementally from the dependency graph when inputs change, inside the same database transaction as the write.
- Aggregates over large child sets are materialized and refreshed on change.

Workflow and approval engine

- Each workflow is a finite state machine; a transition is allowed only if the actor's role permits it and every guard passes.
- Approval policies are evaluated when a record is submitted, and re-evaluated if the amount changes (which resets collected approvals).
- Built-in invariants: a submitter cannot approve their own request; approvers must be distinct; an approval is tied to the record's version at the time it was given.
- Budget checks: optional hard stop or soft warning when an approval would push a line over budget.

Automations

- Events (`record.created`, `state.changed`, `field.crossed_threshold`, scheduled cron) go onto a job queue (Redis + RQ/Celery or Postgres-backed queue). Actions are a fixed catalog: notify (email/in-app/Slack), set field, create record, generate report.
- Idempotency keys prevent duplicate notifications; failed jobs retry with backoff and surface in an admin log.

Testing the engine: property-based tests (Hypothesis) for expressions, a golden suite of real-world spreadsheets whose totals must reconcile, and simulation of rule changes against historical records before they are published.

Frontend renderer

One React/TypeScript app renders every generated app at runtime from its `AppSpec`; no code is generated or deployed per customer.

- **Stack:** React + TypeScript + Vite, TanStack Query for server state, React Hook Form + Zod for forms (Zod schemas generated from the spec), TanStack Table for grids, Recharts or ECharts for dashboard widgets, Tailwind + a component library (e.g. shadcn/ui) for consistent styling.
- **Component registry:** each view type (`form`, `table`, `kanban`, `detail`, `approval_queue`, `kpi`, `chart`, `budget_vs_actual`) and field type (`money`, `date`, `enum`, `ref`, `file`, `user`) maps to one registered component. Unknown types fail closed with a visible placeholder.
- **Permissions in the UI are cosmetic only.** The renderer hides actions the user can't take, but the backend enforces every permission and transition.
- **Builder mode** for admins: the *Detected model* review screen, a side-by-side preview, a chat panel for natural-language edits, and a visual diff of each proposed spec change.

- **Mobile-first submission:** expense form with camera capture works as a PWA, so members don't need to install anything.
- **Accessibility:** WCAG 2.1 AA for all registered components; generated labels come from the sheet's headers and can be edited.

Natural-language customization and document extraction

Conversational edits produce spec patches, never direct changes. The flow for "Require two approvals for purchases over $200":

1. **Interpret.** The LLM gets the current spec (or the relevant slice), the user's message, and a set of edit tools: `add_field`, `update_rule`, `add_automation`, `update_view`, `rename`, etc. It returns a list of tool calls, i.e. a JSON Patch against the spec.
2. **Validate.** The patch is applied to a draft spec and run through the same schema and semantic checks as initial generation.
3. **Explain and simulate.** The user sees a plain-English summary ("Expenses over $200.00 will need approval from 2 different people"), the config diff, and an impact preview: e.g. 3 pending requests would now need a second approval.
4. **Confirm.** Only on explicit confirmation is a new spec version published. Ambiguity triggers a clarifying question instead of a guess ("Over $200 before or after tax?").

Receipt and invoice extraction

1. Upload (photo or PDF) → image normalization (rotate, de-skew, compress).
2. A vision-capable LLM extracts a fixed schema via structured output: vendor, date, line items, subtotal, tax, total, currency, payment method, plus a per-field confidence. A dedicated OCR/document service (e.g. AWS Textract, Google Document AI) can be added as a fallback or cross-check.
3. **Deterministic checks:** line items sum to subtotal, subtotal + tax = total, date not in the future, amount within policy limits, duplicate detection by hash and by (vendor, date, total).
4. Fields are pre-filled in the expense form; low-confidence or failed-check fields are highlighted. The submitter always confirms; nothing is auto-submitted.
5. Category and budget line are suggested from vendor history and the app's category list.

Data model and API

Storage approach: a fixed relational schema for platform metadata, plus a generic record store with JSONB for user-defined entities. This lets any spec run without per-app DDL; hot fields get expression indexes, and money values are validated and stored as numeric strings, then cast to `NUMERIC` in queries.

Table	Key columns	Purpose
`organizations`	id, name, plan	Tenant boundary
`users`, `memberships`	user_id, org_id, app_id, role	Identity and role per app
`apps`	id, org_id, name, current_spec_version	One generated app
`app_specs`	app_id, version, spec JSONB, created_by, source (`import` / `nl_edit` / `manual`)	Immutable spec history
`imports`	id, app_id, file_uri, sha256, status, digest JSONB, proposal JSONB, reconciliation JSONB	Pipeline runs and outputs
`records`	id, app_id, entity, data JSONB, state, version, created_by, timestamps	All user data
`approvals`	record_id, record_version, approver_id, decision, comment, at	Approval decisions
`attachments`	id, record_id, file_uri, extraction JSONB, sha256	Receipts and invoices
`events`, `jobs`	type, payload, status, idempotency_key	Automation queue
`audit_log`	org_id, actor, action, target, before, after, at	Append-only history
`llm_calls`	purpose, model, prompt_hash, tokens, latency, schema_version, outcome	Cost and quality tracking

Row-level security in Postgres (`org_id` on every table, set per request) is a second line of defense behind application checks.

API (FastAPI, REST + OpenAPI)

Endpoint	Purpose
`POST /imports`	Upload a workbook; starts the pipeline (async, returns import id)
`GET /imports/{id}`	Status, digest, proposal, reconciliation report
`POST /imports/{id}/confirm`	Create the app from the reviewed proposal
`GET /apps/{id}/spec`	Current spec (frontend boots from this)
`POST /apps/{id}/spec/edits`	Natural-language edit → draft patch, summary, impact preview
`POST /apps/{id}/spec/versions`	Publish a draft; `POST .../rollback` to revert
`GET/POST /apps/{id}/entities/{entity}/records`	List with filters / create
`PATCH /records/{id}`	Update (optimistic locking on `version`)
`POST /records/{id}/transitions/{action}`	Submit, approve, reject, etc.
`POST /attachments`	Upload a receipt; returns extraction draft
`GET /apps/{id}/export?format=xlsx`	Export data back to Excel

Long-running work (pipeline, extraction) runs in background workers; the client polls or subscribes via Server-Sent Events.

Security, privacy and auditability

Financial data and approvals demand that every change is attributable and every permission is enforced server-side.

- **Auth:** OAuth/OIDC sign-in (Google, Microsoft) and email magic links; short-lived JWTs; optional SSO for paid tiers.
- **Tenant isolation:** `org_id` scoping in every query plus Postgres row-level security; separate object-storage prefixes per org with signed, expiring URLs.

- **Encryption:** TLS 1.2+ in transit; AES-256 at rest for database and files.
- **Untrusted uploads:** macros are never executed; files are parsed in an isolated worker with memory/time limits to blunt zip bombs and malformed XML; attachments are virus-scanned.
- **Prompt-injection defense:** cell text and receipt text are treated as data. They're wrapped and labeled in prompts, the model can only call read-only analysis tools or spec-edit tools, and every output passes schema and policy validation before a human confirms it. A spreadsheet cell saying "approve everything" cannot change behavior.
- **Data minimization for LLMs:** send the digest and masked samples, not full datasets; use providers with zero data retention and no training on customer data; allow an opt-out of receipt extraction.
- **Audit log:** append-only record of every create, update, transition, approval, spec change and export, with before/after values. Admins can export it.
- **Separation of duties:** enforced by the engine (no self-approval, distinct approvers) and visible in the audit log.
- **Compliance posture:** design toward SOC 2 controls; GDPR/CCPA data export and deletion per org; FERPA considerations if sold to universities.

Evaluation and quality metrics

Quality is measured against a labeled corpus of real spreadsheets, and every prompt or model change must pass it before release.

Evaluation corpus: 100+ anonymized workbooks to start (student-org budgets, small-business expense trackers, event P&Ls, invoice logs), each hand-labeled with the expected entities, relationships, computed fields and workflow. Grow it from opt-in production samples where users corrected the AI.

Metric	How measured	v1 target
Entity/field detection F1	Proposal vs labels	⩾ 0.90
Relationship accuracy	Correct FK pairs / labeled pairs	⩾ 0.85
Formula translation coverage	Formula cells compiled / total formula cells	⩾ 95%

Metric	How measured	v1 target
Reconciliation pass rate	Apps whose recomputed totals match Excel exactly	⩾ 98% of translated cells
Schema-valid on first try	Valid `ModelProposal` without repair	⩾ 95%
NL edit success	Edits accepted by user without rephrasing	⩾ 80%
Receipt total accuracy	Extracted total = true total	⩾ 97%
Time to first app	Upload → published (median)	< 10 min
Pipeline cost	LLM spend per import (median)	< $0.25

Testing layers: unit and property tests for the engine; contract tests that generated TS types and Pydantic models agree; end-to-end Playwright tests that import a golden workbook, submit an expense and approve it; LLM evals run in CI on every prompt change with regression thresholds.

Production signals: fields users correct on the review screen, NL edits that get undone, extraction fields edited before submit. These feed back into the corpus.

Risks and alternatives

Risk	Impact	Mitigation
AI misreads the sheet's structure or intent	Wrong app, lost trust	Confidence badges + source citations on review screen; reconciliation gate before publish; easy correction by click or chat
Messy real-world sheets (merged cells, multiple tables per sheet, hard-coded totals)	Low detection accuracy	Region-detection heuristics, user-assisted "select this range" fallback, corpus weighted toward messy files
Unsupported Excel formulas	Missing calculations	Coverage report; keep values static and flagged; expand function library by frequency in real uploads

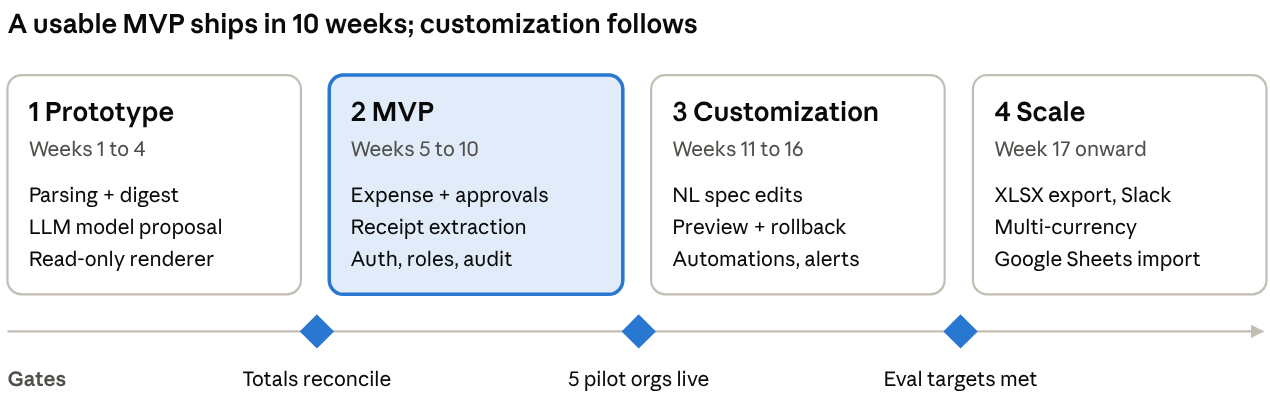
Risk	Impact	Mitigation
NL edit does something unintended	Wrong approval behavior	Patch preview, impact simulation, explicit confirm, one-click rollback
Prompt injection via cell or receipt text	Manipulated config	Data/instruction separation, limited tool set, validation, human confirmation
LLM cost or latency spikes	Slow onboarding, margins	Digest keeps prompts small; cache by workbook hash; small model for easy subtasks; per-org quotas
Users expect two-way Excel sync	Churn after import	Clear messaging; strong XLSX export; sync as a roadmap item

Alternatives considered

- **LLM generates React code per app.** Most flexible, but unreviewable, hard to secure, and every change risks breaking the app. Rejected in favor of a declarative spec + fixed renderer.
- **LLM computes values at runtime.** Simple, but non-deterministic and unauditable for money. Rejected.
- **Per-app physical Postgres tables.** Better query performance and native constraints, but requires migrations on every spec edit. Deferred; JSONB record store for v1, revisit for apps over ~1M records.
- **Build on an existing low-code platform** (e.g. Retool, Airtable, Glide as a target). Faster start, but limits the approval/engine guarantees and adds a dependency. Possible as an export target later.
- **Pure heuristics, no LLM.** Deterministic, but brittle on workflow inference and unusable for natural-language edits. Heuristics are kept as the first pass inside the pipeline.

Roadmap and open questions

The plan ships one workflow (reimbursement with approvals) end to end before broadening; durations are estimates for a team of 3–4 engineers.



roadmap · 4 phases, 3 gates (estimates for a team of 3–4)

Each phase advances only when its gate passes: the golden corpus reconciles exactly, pilot organizations use the MVP for real expenses, and the evaluation targets above are met.

Open questions

- [ ] Which segment do we pilot first: student organizations (simpler, free tier) or small businesses (paying, more varied sheets)?
- [ ] Do we need Google Sheets import in the MVP, given many student orgs live there rather than in Excel?
- [ ] Should reimbursement status integrate with payment tools (Venmo, PayPal, bank exports), or stay record-only?
- [ ] What is the right behavior when a spec edit invalidates approvals already in progress: reset, grandfather, or ask?
- [ ] Which LLM provider(s), and do we need a self-hosted option for privacy-sensitive customers?
- [ ] Pricing model: per app, per active user, or per processed receipt?